



UNINASSAU

LoadForge

Stress and Resilience Evaluator

RECIFE - PE
2026

Planejamento da Sprint 01

LoadForge Stress and Resilience Evaluator

Este documento define o problema, o escopo do produto, os requisitos iniciais e o plano semanal de desenvolvimento do LoadForge. O MVP concentra-se na execução segura de testes de carga e inclui inteligência artificial integrada à otimização do processamento. Um modelo local, treinado pela equipe, estimará o risco de degradação da aplicação, e o controlador adaptativo usará essa previsão para regular a concorrência do motor de carga.

1 Formação da Equipe

Turma: Fábrica de Software 2026.2

Integrante	Papel inicial
Aline Bianca Arantes da Silva	Desenvolvedora Backend Python
Camile Marcele Pereira de Araújo	Product Owner e Analista de QA
Cayo Vitor Fagundes	Desenvolvedor Frontend e UI UX
Emerson Wallace Barcelos de Araújo	Scrum Master e Desenvolvedor Backend
Ricardo Cezar Ottoni Assis de Almeida	Dados infraestrutura e documentação

2 Escolha do Tema

O tema escolhido é uma plataforma web para planejamento, execução e análise de testes de carga em ambientes autorizados. O projeto pertence às áreas de Quality Assurance, confiabilidade de software, inteligência artificial e otimização de processamento. O diferencial técnico está na integração entre um motor assíncrono, um modelo preditivo local e um controlador que regula a concorrência durante o teste.

3 Definição do Problema

Equipes de desenvolvimento nem sempre conseguem medir como uma aplicação se comporta quando o volume de acessos aumenta. Ferramentas existentes podem exigir configuração especializada, e um gerador de carga mal dimensionado pode saturar a própria máquina de teste e distorcer os resultados. Além disso, controladores baseados somente em limites reagem depois que a degradação já começou. O LoadForge pretende reduzir essa dificuldade ao executar cenários de carga controlada, aprender padrões anteriores de degradação e ajustar a concorrência antes que latência e erros ultrapassem os limites definidos pelo usuário.

O problema é relevante porque falhas sob carga afetam disponibilidade, experiência do usuário e capacidade de planejamento. A solução será restrita a aplicações próprias ou expressamente autorizadas, com limites de duração, concorrência e interrupção de emergência.

4 Objetivos do Sistema

Objetivo geral: desenvolver um sistema web que permita configurar, executar e comparar testes de carga, com persistência dos resultados, previsão de degradação por inteligência artificial e otimização adaptativa do motor gerador.

Objetivo específico 1: implementar um motor de requisições HTTP concorrentes em Python com programação assíncrona e limites configuráveis.

Objetivo específico 2: construir um conjunto de dados próprio a partir de execuções controladas e treinar localmente um modelo que estime o risco de degradação nos próximos segundos.

Objetivo específico 3: integrar a previsão do modelo a um controlador que ajuste a concorrência a partir do risco previsto e das métricas observadas, sem depender de API externa.

Objetivo específico 4: comparar carga fixa, otimização por regras e otimização assistida por IA com métricas de desempenho, eficiência e qualidade preditiva.

Objetivo específico 5: oferecer autenticação, perfis de acesso, cadastro de projetos e endpoints e histórico dos testes executados.

4.1 Escopo do MVP

Escopo	Definição
Incluído no MVP	Login com perfis QA e Visualizador; projetos e endpoints; motor assíncrono; coleta de dados por janelas; treinamento e validação de modelo local; previsão de risco durante o teste; controlador híbrido com IA; comparação entre três estratégias; persistência, relatórios, testes e instruções de execução.
Fora do MVP	Chatbot ou IA generativa; consumo de API externa de IA; seleção autônoma de alvos; assinaturas e cobrança; arquitetura multi-tenant; cluster em várias máquinas; aplicativo móvel; plugins; relatórios comerciais avançados; integração com Celery.

4.2 Inteligência Artificial e Otimização

O componente avançado será formado pelo DegradationRiskModel e pelo AdaptiveLoadController. O primeiro estimará a probabilidade de degradação da aplicação; o segundo combinará essa previsão com as métricas observadas para ajustar a concorrência do motor. Todo o processamento ocorrerá localmente e será integrado ao fluxo principal do sistema.

4.2.1 Modelo de Previsão de Degradação

Durante execuções controladas, o sistema agrupará os dados em janelas de dois segundos. Cada amostra poderá incluir concorrência atual, throughput, latências p50, p95 e p99, taxa de erro, timeouts, CPU, memória e tendências recentes dessas medidas. A amostra será rotulada como degradação próxima quando, nos dez segundos seguintes, ocorrer violação do limite de latência, aumento relevante de erros ou queda de throughput mesmo com aumento de concorrência.

A equipe implementará o pipeline de preparação dos dados, criação dos rótulos, treinamento, validação, seleção e versionamento do modelo. Serão comparados ao menos um modelo linear, como regressão logística, e um modelo baseado em árvores, como Random Forest, usando Python e scikit-learn. A escolha considerará F1, recall, falsos positivos e tempo de inferência. O modelo escolhido será salvo e executado localmente; nenhuma API externa de IA será utilizada.

4.2.2 Controlador Híbrido de Concorrência

O AdaptiveLoadController receberá, a cada janela, a probabilidade de degradação produzida pelo modelo e as métricas atuais. Os limites iniciais serão configuráveis e poderão ser ajustados após os experimentos.

- Risco baixo e métricas estáveis: aumentar gradualmente a concorrência dentro do limite definido.
- Risco intermediário: manter a concorrência e aguardar a próxima janela de observação.
- Risco alto ou violação observada: reduzir a concorrência e registrar o motivo da decisão.
- Modelo indisponível ou entrada inválida: retornar automaticamente ao controlador baseado em regras.

Cada decisão armazenará horário, versão do modelo, probabilidade prevista, métricas observadas, estratégia ativa e novo valor de concorrência. Esse registro permitirá explicar a atuação da IA e reproduzir o experimento.

4.2.3 Fluxo de Integração

Etapa	Funcionamento
1 Coleta	O motor assíncrono envia requisições e agrega as métricas da janela.
2 Preparação	O sistema calcula tendências e monta o vetor de características.
3 Inferência	O modelo local retorna a probabilidade de degradação.
4 Otimização	O controlador aumenta, mantém ou reduz a concorrência.
5 Registro	Métricas, previsão, decisão e versão do modelo são persistidas.
6 Relatório	A interface compara as estratégias e apresenta a atuação da IA.

4.3 Métricas e Protocolo de Avaliação

Métricas de processamento: avaliam o motor e o controlador.

Métrica	Cálculo	Finalidade
Throughput	Requisições concluídas com sucesso por segundo	Mede a capacidade do gerador
Latência p50 p95 p99	Percentis do tempo de resposta	Mostra comportamento típico e cauda de latência
Taxa de erro	Percentual de timeouts e respostas com erro	Verifica se o ganho compromete a estabilidade
CPU e memória do gerador	Consumo médio e pico durante o teste	Detecta saturação da máquina que gera a carga
Eficiência	Throughput dividido pelo uso médio de CPU	Compara uso de recursos entre os modos
Estabilidade do controle	Quantidade e amplitude das alterações de concorrência	Identifica oscilações excessivas

Métricas de inteligência artificial: avaliam a capacidade preditiva e o custo de inferência.

Métrica	Cálculo	Finalidade
Precisão	Proporção das previsões de risco que antecedem degradação	Mede a confiabilidade dos alertas
Recall	Proporção das degradações antecipadas pelo modelo	Mede a capacidade de não perder eventos
F1	Média harmônica entre precisão e recall	Métrica principal para seleção do modelo
Falsos positivos	Alertas sem degradação posterior	Evita reduções desnecessárias de carga
Antecedência	Tempo entre o alerta e a degradação observada	Mostra utilidade operacional da previsão
Latência de inferência	Tempo para gerar uma previsão	Confirma que a IA não atrasa o controle

Os dados serão separados cronologicamente em conjuntos de treino, validação e teste para reduzir vazamento entre execuções. Cada cenário final será repetido pelo menos três vezes no mesmo ambiente e com os mesmos limites, comparando: modo fixo, controlador por regras e controlador híbrido com IA. A equipe usará a mediana dos resultados.

As metas iniciais são F1 de pelo menos 0,75, recall de pelo menos 0,80 e inferência inferior a 100 milissegundos por janela. Para a otimização, espera-se ganho de pelo menos 10 por cento em throughput ou redução de pelo menos 10 por cento em CPU, latência p95 ou taxa de erro, sem piorar outra métrica crítica em mais de 10 por cento. Resultados inferiores às metas serão documentados e usados para revisar dados, rótulos, modelo e regras de controle.

5 Público Alvo

O LoadForge será utilizado principalmente por:

- profissionais de QA e testes que precisam reproduzir cenários de carga;
- desenvolvedores backend que investigam gargalos e regressões;
- profissionais de SRE e DevOps que acompanham capacidade e resiliência;
- gestores técnicos que precisam consultar resultados consolidados.

6 Requisitos Funcionais

ID	Requisito
RF01	Permitir cadastro, login e logout com os perfis QA e Visualizador.
RF02	Permitir ao perfil QA cadastrar, editar e remover projetos e endpoints autorizados.
RF03	Permitir configurar método HTTP, duração, concorrência inicial, limite máximo, ramp-up, timeout e estratégia de controle.

ID	Requisito
RF04	Exigir confirmação de autorização do alvo antes da execução e oferecer interrupção de emergência.
RF05	Executar requisições concorrentes pelo motor assíncrono, respeitando os limites configurados.
RF06	Coletar por janelas as métricas e tendências necessárias para formar o conjunto de dados da IA.
RF07	Permitir executar o pipeline de preparação, rotulagem, treinamento e validação do modelo local.
RF08	Carregar uma versão aprovada do modelo e estimar o risco de degradação durante o teste.
RF09	No modo híbrido, ajustar a concorrência a partir do risco previsto e das métricas observadas.
RF10	Retornar ao controle por regras quando o modelo estiver indisponível ou receber dados inválidos.
RF11	Persistir configurações, janelas de métricas, previsões, decisões, versão do modelo e resultados.
RF12	Comparar os modos fixo, adaptativo por regras e adaptativo com IA em um relatório único.
RF13	Permitir ao perfil Visualizador consultar projetos, execuções e relatórios sem alterar dados.

7 Requisitos Não Funcionais

ID	Requisito
RNF01	O backend e o motor de carga devem ser desenvolvidos em Python com FastAPI e programação assíncrona.
RNF02	O pipeline de IA deve ser desenvolvido em Python, executado localmente e não pode depender de serviço ou API externa de IA.
RNF03	Dados, parâmetros, semente, versão do modelo e resultados de avaliação devem ser registrados para permitir reprodução.
RNF04	A separação de treino, validação e teste deve respeitar a ordem temporal das execuções para reduzir vazamento de dados.
RNF05	A inferência deve ocorrer em até 100 milissegundos por janela no ambiente de referência da equipe.
RNF06	O controlador deve continuar operando por regras se a inferência falhar ou o modelo não estiver disponível.

ID	Requisito
RNF07	O sistema deve limitar concorrência, conexões, duração e timeout para evitar consumo não controlado de recursos.
RNF08	As métricas devem usar relógio monotônico e unidades documentadas para permitir repetição dos testes.
RNF09	Senhas devem usar hash seguro; segredos não podem ser versionados; rotas devem aplicar autorização por perfil.
RNF10	A execução deve ocorrer apenas após declaração de autorização do alvo e deve possuir cancelamento imediato.
RNF11	O PostgreSQL deve persistir usuários, projetos, endpoints, configurações, métricas, previsões e decisões.
RNF12	A interface deve mostrar status, risco previsto, estratégia ativa, decisões e gráficos legíveis.
RNF13	O código deve incluir testes do pré-processamento, interface do modelo, controlador e fluxo integrado.
RNF14	O repositório deve conter documentação técnica, modelo do banco, descrição do conjunto de dados, ficha do modelo e instruções de execução.

8 Casos de Uso

Ator	Caso de uso	Descrição
QA e Visualizador	Autenticar-se	Acessa o sistema conforme o perfil cadastrado.
QA	Gerenciar projeto e endpoint	Mantém os dados do sistema autorizado que será avaliado.
QA	Configurar teste	Define carga, duração, limites, timeout e estratégia fixa, por regras ou com IA.
QA	Executar e interromper teste	Inicia o motor, acompanha o status e pode acionar a parada de emergência.
Equipe técnica	Treinar e validar modelo	Prepara o conjunto de dados, treina candidatos, avalia e aprova uma versão local.
QA	Executar controle com IA	Executa o teste com previsão de risco integrada ao controlador.
QA	Comparar estratégias	Compara carga fixa, controle por regras e controle assistido por IA.

Ator	Caso de uso	Descrição
QA e Visualizador	Consultar relatório	Visualiza métricas, risco previsto, decisões, versão do modelo e histórico.

9 Product Backlog

ID	Item	Prioridade	Entrega	Critério de aceite
PB01	Definir arquitetura, modelo de dados e ambiente do projeto	Alta	MVP	Diagrama, esquema inicial e projeto executando localmente
PB02	Implementar autenticação e perfis QA e Visualizador	Alta	MVP	Acesso e permissões validados por testes
PB03	Implementar projetos e endpoints	Alta	MVP	CRUD persistido com validação de URL
PB04	Implementar configuração e validação do cenário	Alta	MVP	Limites, duração, timeout e autorização registrados
PB05	Implementar motor assíncrono em modo fixo	Alta	MVP	Execução concorrente com cancelamento e métricas
PB06	Coletar janelas de métricas e formar conjunto de dados	Alta	MVP	Amostras persistidas com origem e horário
PB07	Implementar rotulagem e preparação dos dados	Alta	MVP	Pipeline reproduzível e testado
PB08	Treinar e comparar modelos candidatos	Alta	MVP	Regressão logística e Random Forest avaliadas
PB09	Versionar e carregar o modelo selecionado	Alta	MVP	Modelo e metadados disponíveis localmente
PB10	Implementar controlador adaptativo por regras	Alta	MVP	Decisões testadas e registradas por janela
PB11	Integrar previsão de risco ao controlador	Alta	MVP	Execução híbrida de ponta a ponta
PB12	Criar fallback seguro sem IA	Alta	MVP	Falha do modelo não interrompe o teste
PB13	Criar relatório e comparação entre três estratégias	Alta	MVP	Métricas de IA e processamento apresentadas
PB14	Aplicar segurança, testes e documentação técnica	Alta	MVP	Fluxos críticos testados e documentação reproduzível

ID	Item	Prioridade	Entrega	Critério de aceite
PB15	Preparar vídeos e apresentação final	Média	Final	Vídeos demonstram IA integrada ao sistema

10 Cronograma Inicial

As entregas foram distribuídas em ciclos semanais até 05/12. Em cada orientação, a equipe apresentará o repositório atualizado, as funcionalidades concluídas, as dificuldades encontradas e o plano da semana seguinte.

Ciclo	Data	Objetivo semanal	Evidência esperada
Sprint 01	05/09	Tema, problema, requisitos, casos de uso, backlog e planejamento inicial	Documento da Sprint 01 e repositório criado
Sprint 02	19/09	Revisão do escopo, arquitetura de IA, modelo de dados e protótipo	Fluxo IA e otimização validado
Sprint 03	19/09	Configuração do backend, banco, frontend e integração contínua	Ambiente reproduzível e esquema inicial
Sprint 04	26/09	Autenticação e perfis de acesso	Login e autorização por perfil
Sprint 05	03/10	Projetos, endpoints e regras de autorização do alvo	CRUD e validações funcionando
Sprint 06	17/10	Coleta por janelas, persistência, rotulagem e conjunto de dados	Dataset inicial documentado
Sprint 07	24/10	Treinamento, validação e seleção do modelo local	Modelo versionado com métricas
Sprint 08	31/10	Controlador adaptativo por regras e fallback	Controle sem IA testado
Sprint 09	07/11	Integração da inferência local com o controlador	Execução híbrida de ponta a ponta
Sprint 10	14/11	Relatório de risco, decisões e comparação das estratégias	Atuação da IA visível e explicável
Sprint 11	21/11	Experimentos em três modos, ajustes e testes de integração	Três repetições por cenário registradas
Sprint 12	28/11	Segurança, documentação, ficha do modelo, QA e vídeos	Versão candidata à entrega
Entrega final	05/12	Entrega e apresentação	Sistema, IA, otimização, código, banco, documentação e vídeos

11 Repositório GitHub

O repositório base da equipe está disponível em:

<https://github.com/camimcl/sre-fabrica-de-software>

O repositório deverá manter o código-fonte organizado, histórico de commits, documentação técnica, modelo do banco de dados, descrição do conjunto de dados, pipeline de treinamento, ficha da versão do modelo, testes e instruções de instalação e execução.

PARTE II SPRINT 02

Arquitetura, modelagem, prototipação e estrutura técnica

A Sprint 02 estabelece a base técnica do LoadForge para o desenvolvimento do MVP. A arquitetura, os modelos, o protótipo e o banco de dados descrevem o mesmo fluxo entre execução autorizada, métricas, inteligência artificial local e otimização adaptativa.

Materiais do projeto: [repositório GitHub](#) | [protótipo no Figma](#) | [fontes e imagens dos diagramas](#)

12 Arquitetura do Sistema

O LoadForge será desenvolvido como um monólito modular. A aplicação web React consumirá uma API REST FastAPI. O backend concentrará autenticação, projetos, cenários, execução da carga, métricas, relatórios, treinamento e inferência. O PostgreSQL armazenará dados transacionais e experimentais, enquanto os artefatos dos modelos serão versionados localmente. Essa arquitetura reduz a complexidade operacional do MVP e preserva separação entre os módulos por contratos explícitos.

Componente	Tecnologia	Responsabilidade e comunicação
Frontend web	React, Vite e TypeScript	Consome a API REST; apresenta configuração, monitoramento e relatórios.
API e módulos	Python 3.11 e FastAPI	Valida comandos, aplica regras de negócio e coordena persistência, IA e controlador.
Motor de carga	asyncio e httpx	Executa requisições concorrentes autorizadas, com limites, timeout e cancelamento.
Métricas	Janelas de 2 segundos	Agrega throughput, latências, erros, timeouts, CPU e memória.
IA local	scikit-learn e joblib	Treina, valida, versiona e executa o DegradationRiskModel sem API externa.
Controlador adaptativo	Python	Combina risco previsto e métricas; aumenta, mantém ou reduz a concorrência com fallback por regras.
Persistência	PostgreSQL 16	Guarda usuários, cenários, execuções, métricas, previsões, decisões e relatórios.

12.1 Integração do componente computacional avançado

- Cada MetricWindow é enviada ao modelo ativo para estimar a probabilidade de degradação nos dez segundos seguintes.
- O AdaptiveLoadController usa a previsão e as métricas observadas para decidir a nova concorrência.
- Se o modelo estiver indisponível ou a entrada for inválida, o RulesFallbackController assume a decisão.
- RiskPrediction e ControlDecision são persistidos com versão do modelo e justificativa, permitindo auditoria e comparação.

12.2 Diagrama de arquitetura

O diagrama é lido em três faixas: acesso e execução autorizada, ciclo adaptativo e persistência. No ciclo central, as métricas alimentam a previsão local, a previsão orienta o controle híbrido e a nova concorrência retorna ao orquestrador. O monólito modular foi escolhido para reduzir a complexidade operacional do MVP, mantendo responsabilidades separadas e todo o ciclo crítico no mesmo processo. As ligações ortogonais deixam explícita a sequência sem cruzamentos.

Arquivos editáveis e imagens: [docs/diagrams no repositório](#)

Arquitetura do LoadForge

Leitura em três faixas: acesso, ciclo adaptativo e persistência

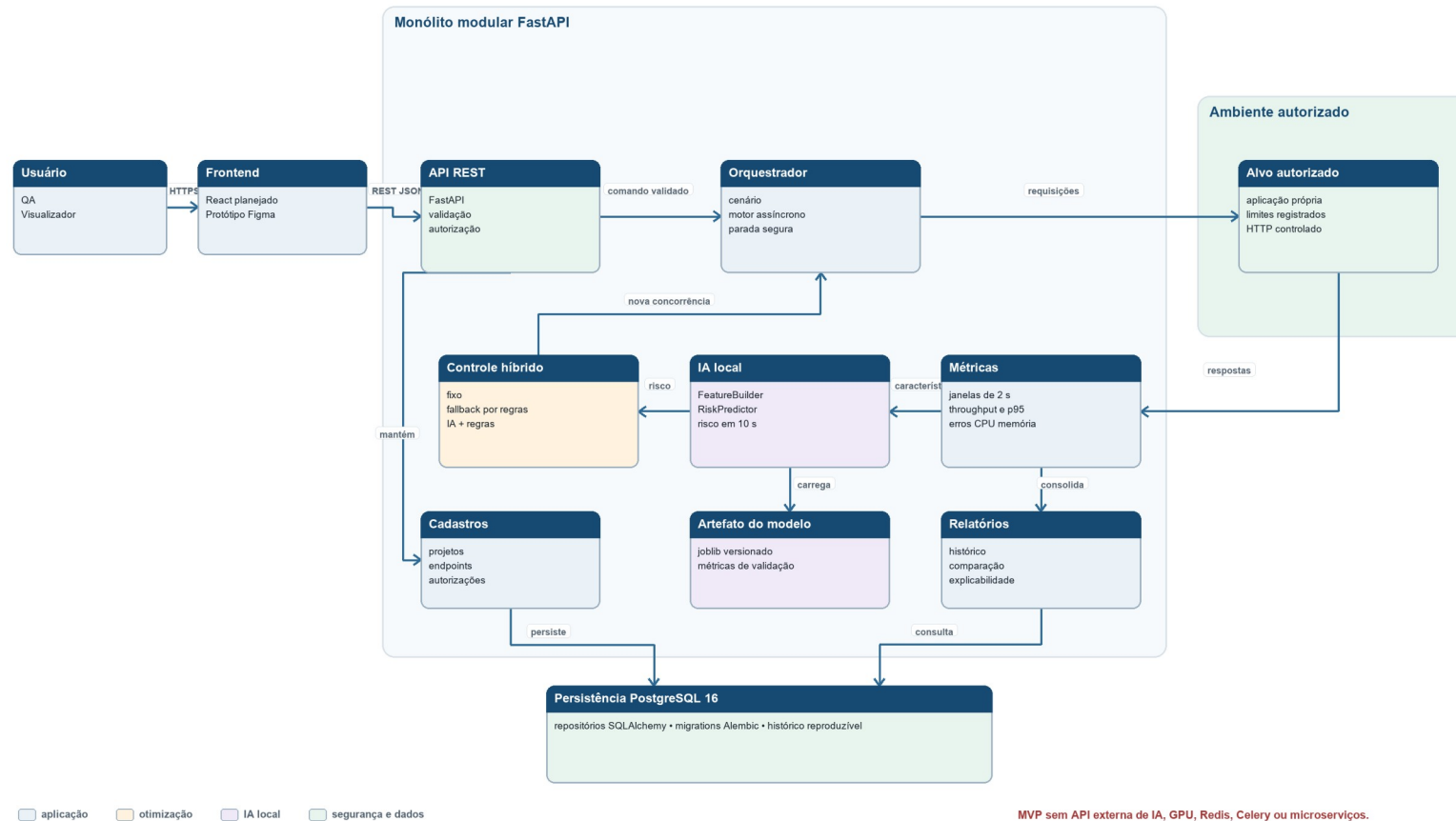


Figura 1 — Arquitetura geral e integração entre software, IA local e otimização adaptativa.

13 Diagrama de Classes

O diagrama foi dividido em dois painéis. O primeiro organiza as entidades persistidas e sua cadeia principal; o segundo apresenta serviços, contratos, IA e controle. Os contratos RiskPredictor e ConcurrencyController evitam acoplamento aos algoritmos, permitem substituir implementações e mantêm o fallback por regras testável de forma independente. Referências secundárias permanecem como atributos UUID para conservar a rastreabilidade sem criar cruzamentos visuais.

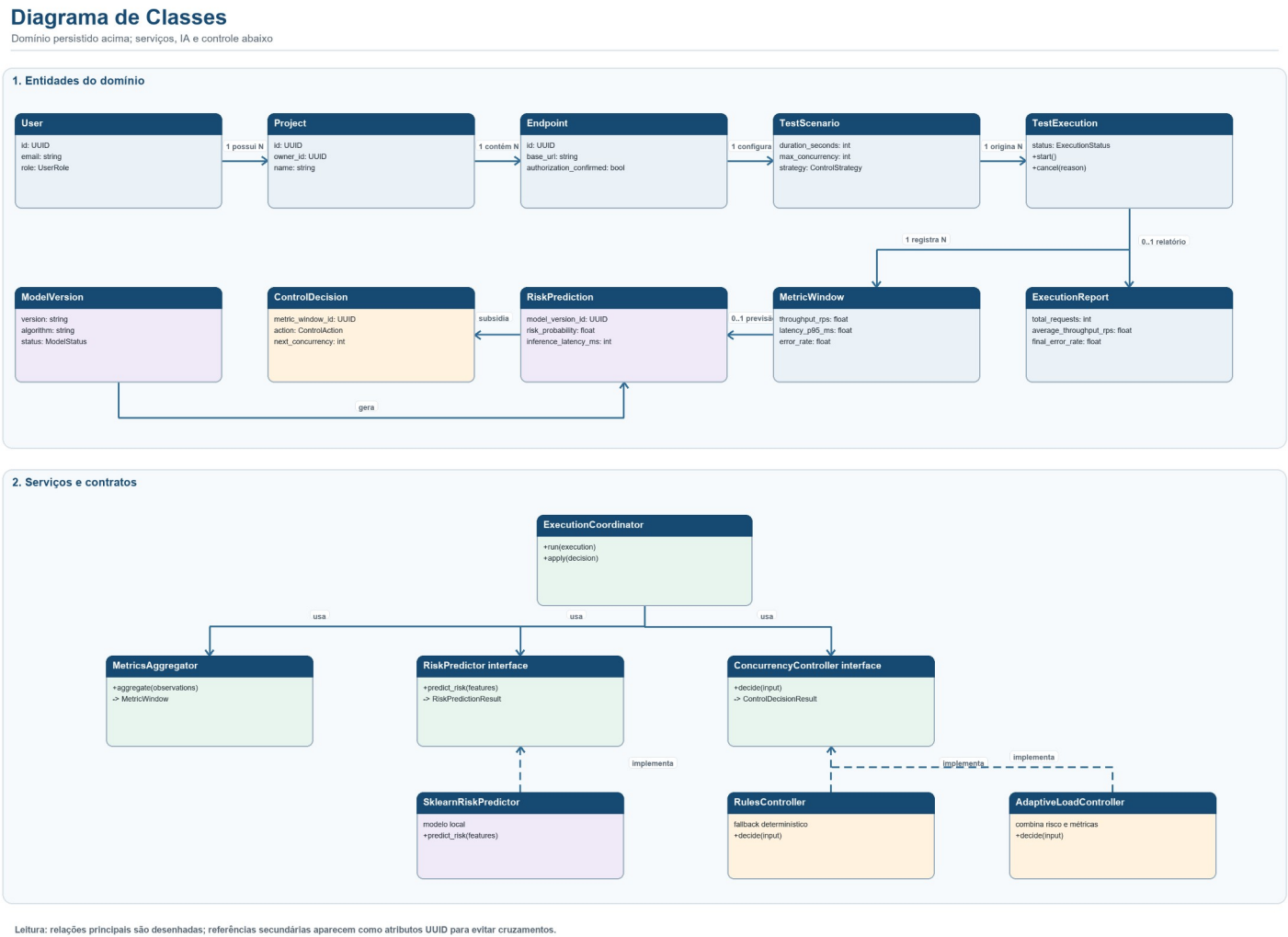


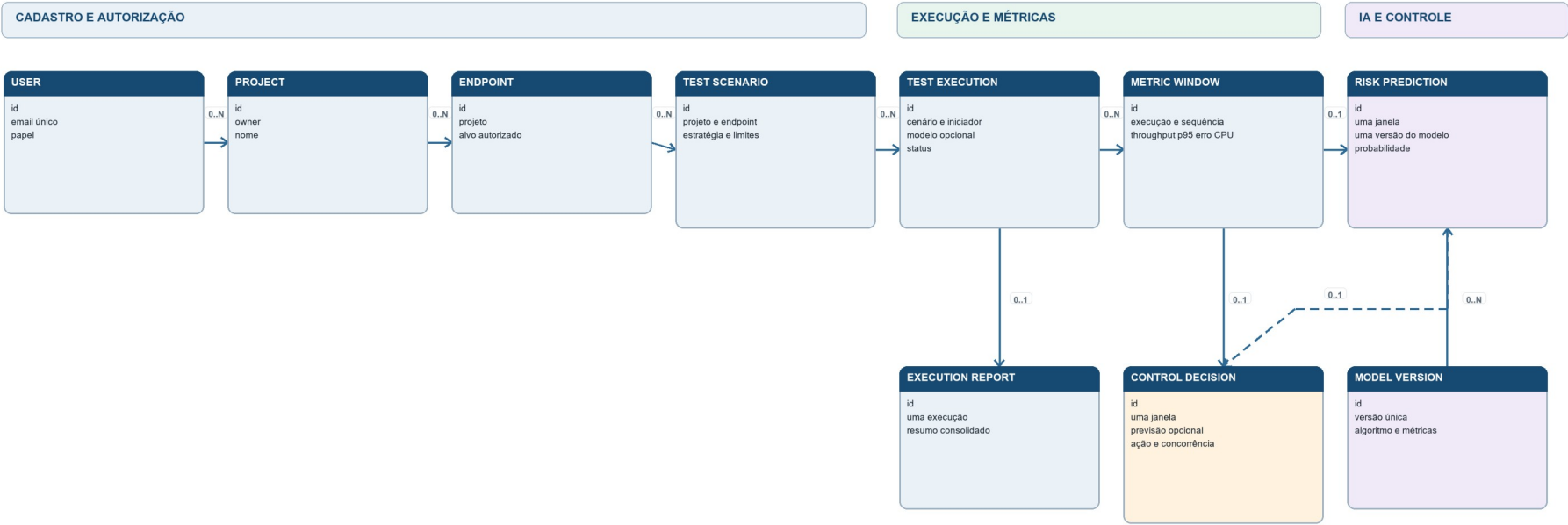
Figura 2 — Diagrama de classes com entidades, serviços e contratos do componente avançado.

14 Modelo Entidade-Relacionamento (MER)

O MER apresenta uma cadeia principal da esquerda para a direita, agrupada em cadastro e autorização, execução e métricas, e IA e controle. Relatório, decisão e versão do modelo aparecem abaixo da entidade de origem. Resultados são opcionais porque uma execução ou janela pode existir antes da conclusão do processamento. A linha tracejada identifica que uma decisão pode usar uma previsão ou ser produzida pelo fallback por regras.

Modelo Entidade-Relacionamento

Cadeia principal da esquerda para a direita; extensões abaixo da entidade de origem



Rótulo: quantidade possível no destino para cada registro de origem. Linha tracejada: vínculo opcional.
Relações secundárias de autoria, projeto e versão do modelo permanecem descritas nos atributos das entidades.

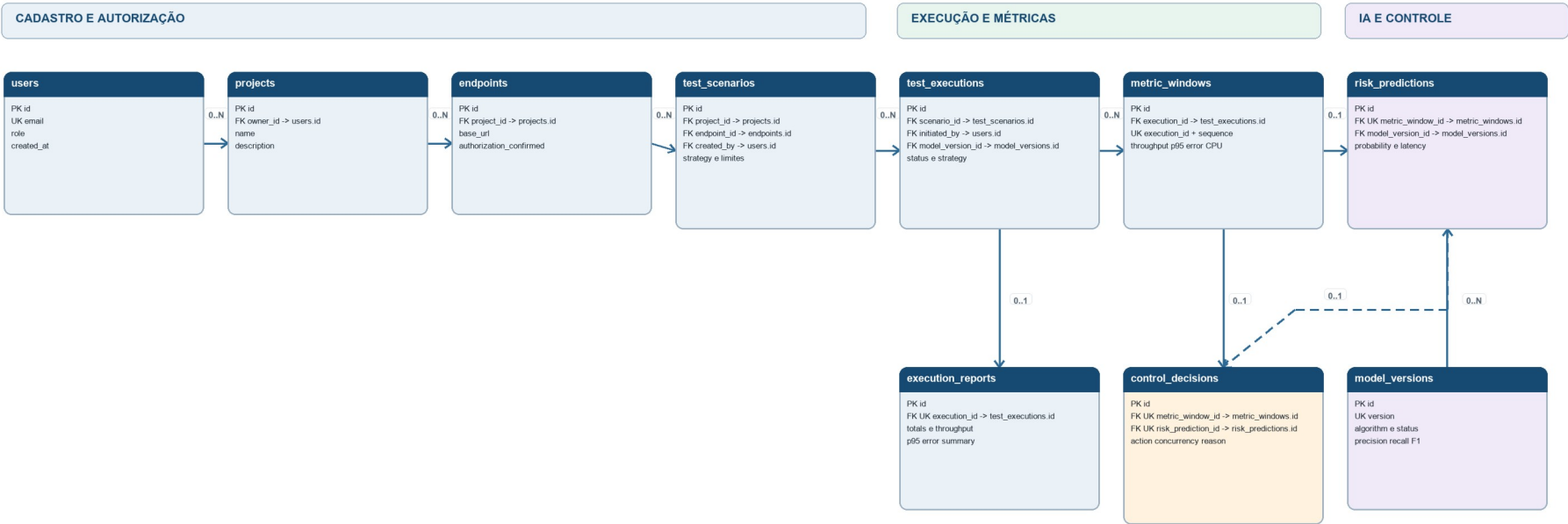
Figura 3 — MER conceitual do banco de dados.

15 Modelo Relacional

A conversão relacional usa UUIDs como chaves primárias para desacoplar a identificação dos módulos. Chaves estrangeiras preservam autoria e origem dos dados; restrições únicas limitam previsão e decisão a uma por janela e relatório a um por execução. Projetos podem existir sem endpoints, execuções sem métricas e janelas sem decisão enquanto o processamento não foi concluído. O caminho principal recebe linhas, e as demais referências aparecem dentro das tabelas como FK → tabela.id. A migration Alembic é a fonte executável do esquema e database/schema.sql apresenta a representação SQL equivalente.

Modelo Relacional

Chaves e referências explícitas; somente o caminho principal recebe linhas



Rótulo: quantidade possível no destino para cada registro de origem. Linha tracejada: vínculo opcional.
As demais relações são mostradas dentro das tabelas como FK -> tabela.id; isso mantém o diagrama completo sem linhas sobrepostas.
Fonte executável: migration Alembic. Representação SQL equivalente: database/schema.sql.

Figura 4 — Modelo relacional implementado no PostgreSQL.

16 Protótipo das Telas Principais

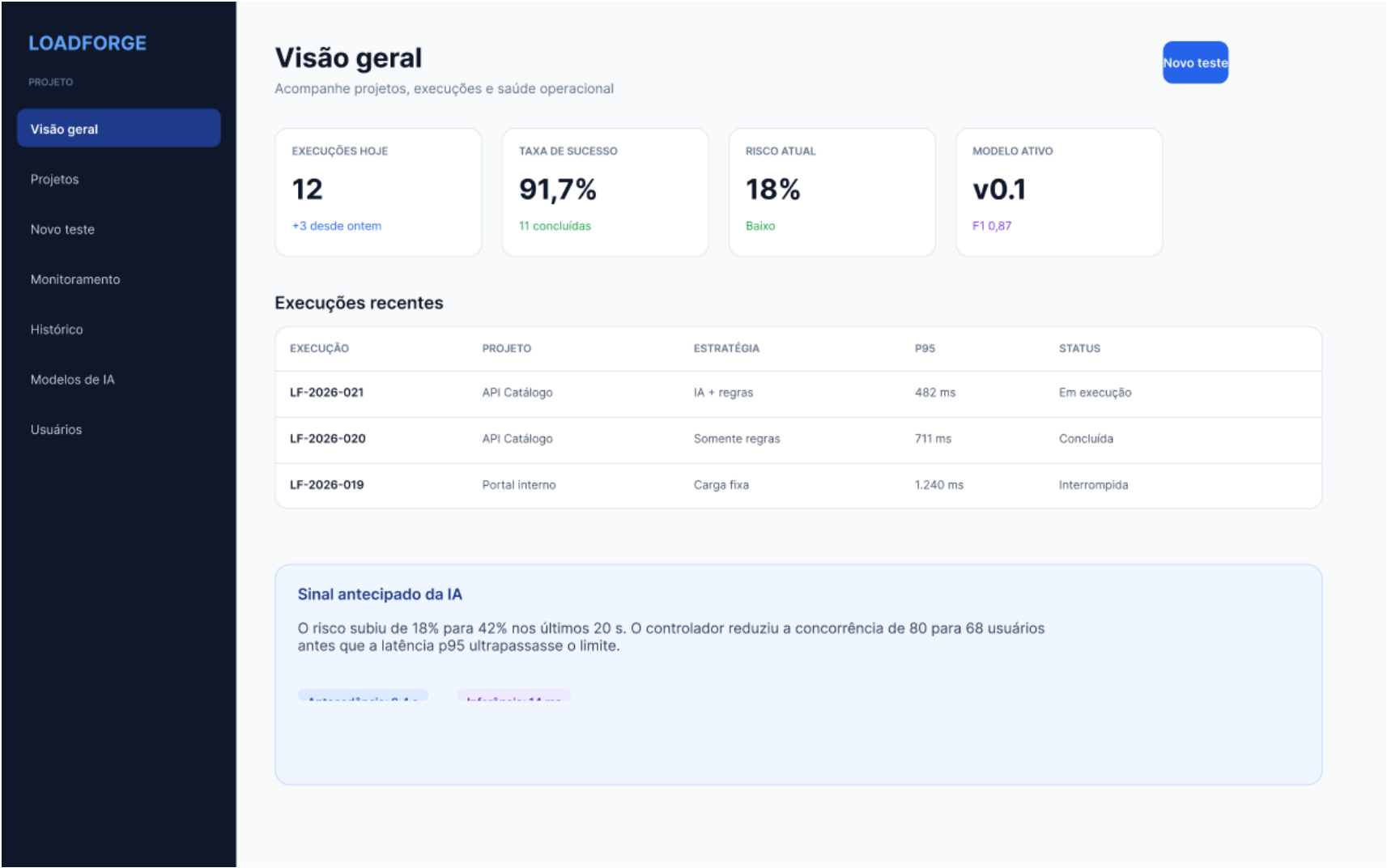
O protótipo inicial foi criado no Figma para validar navegação, hierarquia das informações e visibilidade da IA. Ele serve como referência temporária para a futura interface React; aparência, componentes e distribuição dos elementos poderão mudar durante a implementação e os testes de usabilidade. O fluxo atual prioriza segurança, legibilidade das métricas e compreensão das decisões do controlador.

Arquivo editável: [LoadForge — Protótipo Sprint 02 no Figma](#)

Tela	Objetivo
Login	Autenticar e comunicar que a IA é local e integrada ao produto.
Dashboard	Resumir execuções, risco atual, versão do modelo e sinais antecipados.
Projetos e endpoints	Manter alvos autorizados e seus limites operacionais.
Cadastro de endpoint	Registrar URL, método, limites e evidência de autorização.
Novo cenário	Definir carga, ramp-up, timeout, limites e estratégia de controle.
Monitoramento	Exibir métricas, risco previsto e decisões do controlador em tempo real.
Parada de emergência	Confirmar interrupção segura e preservar as métricas coletadas.
Histórico	Consultar execuções por data, projeto, estratégia e resultado.
Relatório comparativo	Comparar carga fixa, regras e IA com regras em condições equivalentes.
Modelos de IA	Apresentar versões, algoritmo, conjunto de dados e métricas de validação.

16.1 Dashboard

A tela reúne execuções recentes, risco atual e versão ativa do modelo. A síntese foi escolhida para destacar rapidamente sinais de degradação antes da abertura dos detalhes.



16.2 Novo cenário

Carga, limites de segurança e estratégia aparecem no mesmo formulário. A autorização do alvo recebe destaque porque nenhuma execução deve ocorrer sem confirmação prévia.

LOADFORGE

Visão geral

Projetos

Novo teste

Monitoramento

Histórico

Modelos de IA

Usuários

Novo cenário de teste

Configure carga, limites de segurança e estratégia de controle

Parâmetros do cenário

Projeto

API Catálogo

Endpoint

POST /api/search

Estratégia

IA + regras (recomendada)

Duração

5 minutos

Concorrência inicial

20 usuários

Concorrência máxima

100 usuários

Ramp-up

10 usuários / 15 s

Timeout

2.000 ms

Limite de erro

5%

Limite p95

900 ms

Risco para reduzir carga

≥ 65%

Confirmação de autorização

Declaro que o endpoint pertence à equipe ou possui autorização expressa para teste de carga.

CONFIRMAR

Salvar rascunho

Iniciar teste

Figura 6 — Protótipo da tela Novo cenário.

16.3 Monitoramento

Métricas observadas, risco previsto e decisões do controlador compartilham o mesmo contexto. Essa organização relaciona cada ajuste de concorrência aos sinais que o motivaram e mantém a parada imediata acessível.

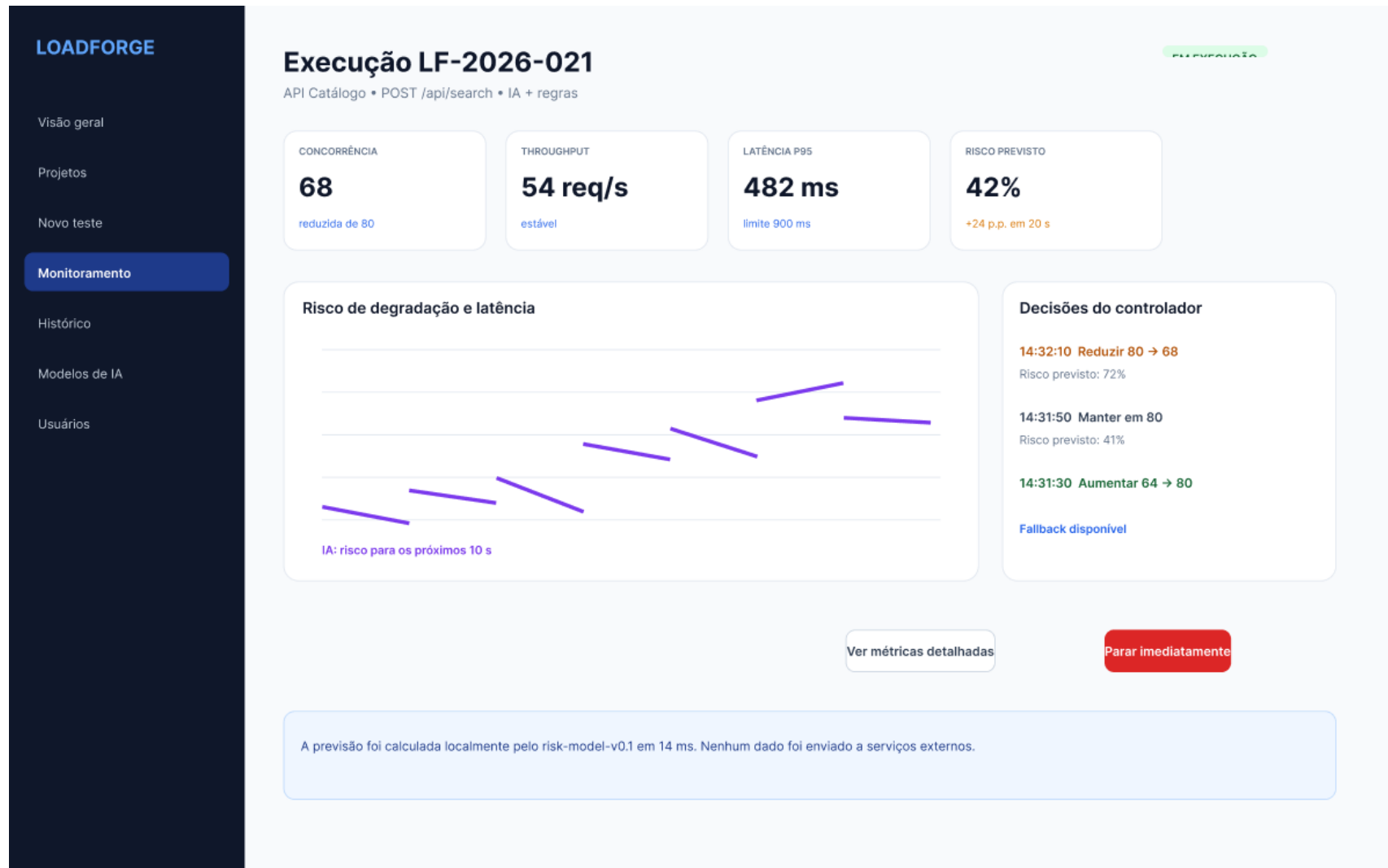


Figura 7 — Protótipo da tela Monitoramento.

16.4 Histórico

A lista permite localizar execuções por estratégia e resultado e acessar a comparação entre os modos fixo, por regras e assistido por IA.

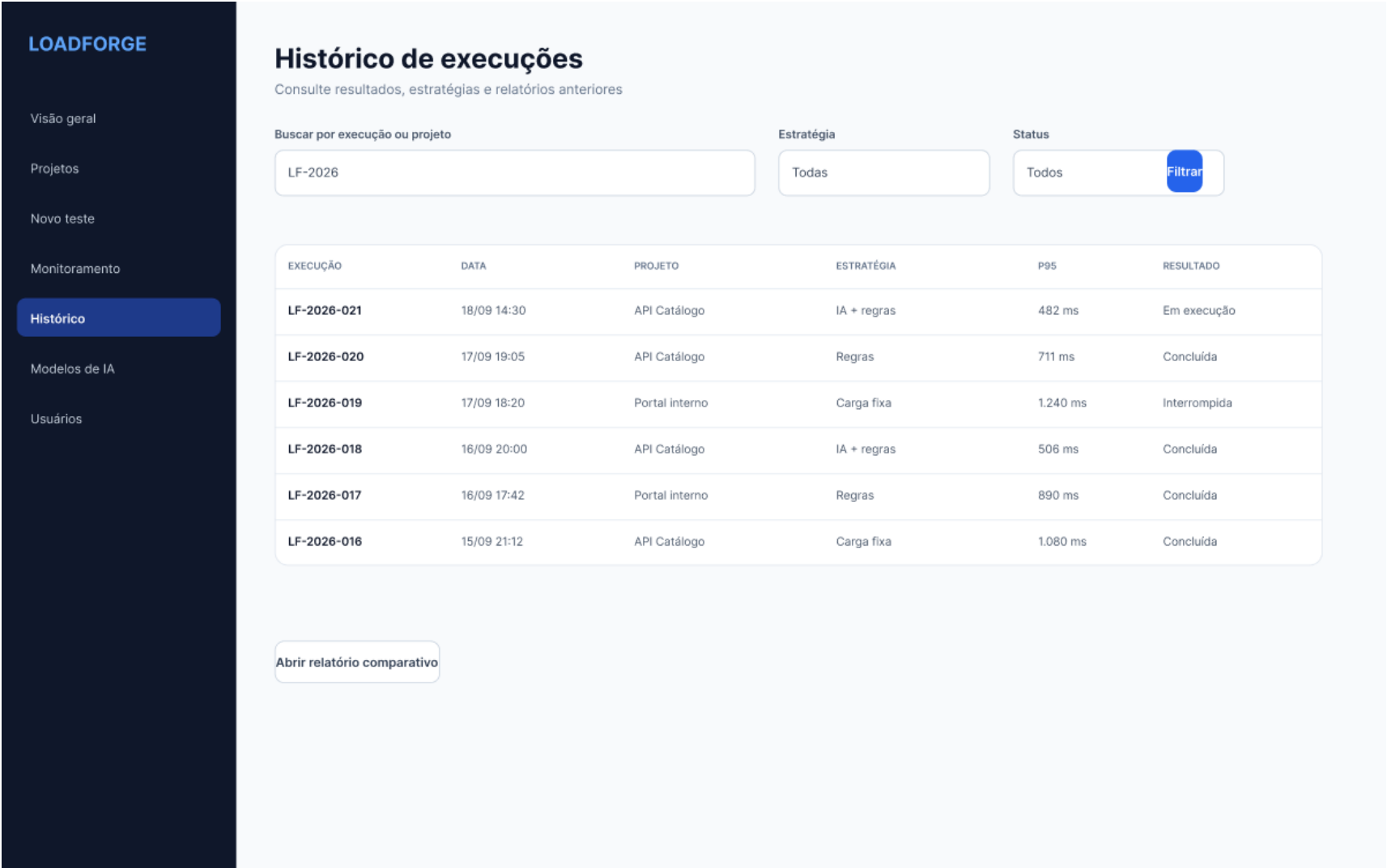


Figura 8 — Protótipo da tela Histórico.

17 Banco de Dados Criado

A estrutura inicial foi implementada para PostgreSQL 16 com SQLAlchemy e Alembic. O repositório contém o esquema SQL de referência, dados iniciais de desenvolvimento e uma migração reversível. As dez tabelas correspondem ao modelo relacional, incluindo as mesmas chaves estrangeiras, relações opcionais e restrições únicas.

Tabela	Finalidade
users	Usuários, senha derivada e papel de acesso.
projects	Projetos de teste e responsável.
endpoints	Alvos autorizados, método, limites e evidência.
test_scenarios	Configuração de carga e limites de segurança.
model_versions	Versões do modelo, algoritmo, artefato e métricas.
test_executions	Execuções, estratégia ativa, status e motivo de interrupção.
metric_windows	Métricas agregadas em janelas de dois segundos.
risk_predictions	Probabilidade, horizonte e latência de inferência.
control_decisions	Ação, concorrência anterior/nova e justificativa.
execution_reports	Resumo consolidado e comparação da execução.

18 Projeto Estruturado no GitHub

Repositório: github.com/camimcl/sre-fabrica-de-software

O repositório foi organizado como monorepo, com separação entre backend, futura interface, banco de dados, diagramas, protótipos e decisões arquiteturais. Essa divisão mantém no mesmo projeto os artefatos que evoluem juntos, enquanto cada pasta possui uma responsabilidade identificável. O README concentra a preparação do ambiente e as orientações operacionais.

Relatório técnico no repositório: [docs/relatorio-sprint-02.md](https://github.com/camimcl/sre-fabrica-de-software/blob/main/docs/relatorio-sprint-02.md)